🚀 Codex SDD Workflow

> **让 Codex 不再"失忆"——一条命令，把可恢复、可审计、可持续迭代的工程工作流注入你的代码仓库。**

开源仓库：https://github.com/formal-you/codex-sdd-workflow

你是否遇到过这些困境？

- Codex 每次重新进入仓库，都像"第一次来"，完全忘记上次做到哪了？
- 多人协作时，任务没留下决策记录、验证证据或下一步方向？
- Session 意外断开，之前的进度、风险、分析全部丢失？

Codex SDD Workflow 就是为了解决这些问题而生的。它在你的仓库内建立一套 **持久化工作区**，让 Codex 每次回来都能立即接续上下文——有任务图、有交接文档、有验证脚本、有完整的执行规范。不靠聊天记忆，靠仓库里的文件。

更准确地说，它是一套面向 AI-assisted engineering 的 **SDD（Specification / System Driven Development）workflow**：把需求、scope、task、验证 evidence、handoff、Git 状态和下一步建议都沉淀为 repo-local artifact。Codex 不再只依赖临时对话，而是按仓库里的规范、脚本和状态文件持续推进。

✨ 一分钟了解核心能力

能力	做了什么	为什么重要
SDD 持久工作区	在仓库中生成 `SDD/` 目录，承载项目简介、架构记录、流程摘要、进度状态	Codex 每次进入仓库都有稳定的起点，不会"从零开始"
Agent 工作契约	自动生成根目录 `AGENTS.md` 和 `README.md` shim	明确 Codex 的启动顺序和行为边界
Agent 底层工具链	提供 `SDD/scripts/` 中稳定确定的操作脚本作为 API	让 AI 代替人类执行管理动作，杜绝 AI 随意修改排版报错，建立强护栏
结构化任务图	Parent task → Subtask 的分层任务系统，含验收标准、验证命令、scope 界定	复杂工程不会变成"一锅粥"，每步可追踪、可验证
Session 生存 & 交接	`session-brief` 脚本 + `handoff` 模板 + `progress.md` 聚合状态 + `state/hot/` 局部便签	断连不丢活，换人不丢线——跨 session 无缝恢复
热状态收敛	引入 `SDD/state/`，把 branch/task 级高频便签从共享 `progress.md` 中拆开	降低多人或多分支协作时的共享写入热点
验证 & 审计链	内置 `validate-sdd`、evidence 模板、workflow audit 模板	确保流程"真的被执行了"，而非停留在纸面
下一步指引	任务结束前必须留下 next-step 记录	每个 session 结束都指向"接下来该做什么"，不会断档

能力	做了什么	为什么重要
Git 完成闭环	按 `TASK_COMPLETION_GIT_MODE` 管理 commit 状态和推荐 message	不会静默 commit，也不会忘记记录"为什么没提交"
双 Profile 灵活选择	`lite` 精准开发 / `full` 敏捷交付 scaffolding	按需选择，从轻量协作到完整 sprint 管理一步切换
连接器预留层	`full` 预留 external issue source 与 pull-only connector hook	能接第三方规划系统，但不把 repo-local 文本误当唯一真源
模板 Overlay	通过 `--template-overlay` 安全覆盖受支持模板	允许团队定制 task / evidence 格式，同时保留 validator 可解析契约
跨平台支持	Shell (Linux/macOS/WSL) 为主，PowerShell 完整支持	Windows、Mac、Linux 团队无缝协作

🧠 设计理念：让 Codex 有稳定的工程记忆

这套 workflow 的核心不是“多生成几个 Markdown 文件”，而是给 Codex 建一套可恢复、可验证、可协作的工程记忆系统。

1. 冷热状态分层

`docs/process.md`、`docs/project-brief.md`、`docs/architecture.md` 是低频变化的冷数据，负责项目背景、长期方向和架构记录。

`docs/progress.md` 是共享恢复摘要，只保留当前阶段、active task 指针、最近验证摘要、推荐下一步和 Git handoff 摘要。

`state/hot/branches/` 与 `state/hot/tasks/` 是高频变化的热数据，承载 branch/task 级临时便签。多人或多分支协作时，Codex 应优先把局部过程写到这里，而不是把 `progress.md` 写成所有人争抢的共享热点。

2. Connector-first，而不是重造项目管理平台

`full` profile 提供 backlog、sprint、release 和 CI/CD scaffolding，但它不是 Jira、GitHub Issues 或 Linear 的替代品。

当团队已经有外部 issue source 时，repo-local artifact 更适合作为执行镜像、handoff 载体和验证 evidence 层。当前 connector 设计默认 `pull-only`：从外部系统拉取上下文，沉淀到本地 workflow，不承诺双向同步或自动回写。

3. Template Overlay：可定制，但不破坏解析契约

内置模板负责保证最低 SDD 结构：checkbox 状态、scope、Acceptance Criteria、Evidence、Completion Handoff、Git handoff。

如果团队需要组织级字段，可以使用 `--template-overlay /path/to/overlay` 覆盖受支持模板。overlay 必须保留 parser contract，不能删除 validator 依赖的标题、checkbox 字段和 handoff 字段。

🎯 谁应该使用它?

- **需要 Codex 长期参与项目演进的开发者** —— 不再每次从头解释背景
- **跨 session 持续推进复杂工程的团队** —— 断连不再是灾难
- **多人 + Codex 混合协作的场景** —— 任务、交接、证据链全部留在仓库里
- **关注工程纪律和可审计性的技术负责人** —— 每一步都有据可查

📦 安装

Linux / macOS / WSL

```
mkdir -p "${CODEX_HOME:-$HOME/.codex}/skills"
git clone <repo-url> "${CODEX_HOME:-$HOME/.codex}/skills/codex-sdd-workflow"
```

Windows PowerShell

```
$env:CODEX_HOME = if ($env:CODEX_HOME) { $env:CODEX_HOME } else { "$HOME\.codex" }
git clone <repo-url> "$env:CODEX_HOME\skills\codex-sdd-workflow"
```

安装完成后，**重启 Codex** 让 skill 列表刷新即可。

🚀 快速上手

安装重启后，用自然语言告诉 Codex 你想做什么——它会自动识别 skill，选择合适的 profile，执行安全预览，然后引导你进入生成的 repo-local workflow。

试试这些提示词

```
"请使用 codex-sdd-workflow 给当前仓库初始化一套 SDD workflow，先预览将要写入的内容。"
```

```
"这个仓库已经有旧版 SDD workflow，请帮我安全升级，不要覆盖根目录已有的 README 和 AGENTS。"
```

```
"我不确定该用 lite 还是 full，请先帮我判断，再接入 workflow。"
```

```
"请为这个多人协作项目接入完整 agile delivery scaffolding。"
```

> 💡 **Tip**: 如果你没有明确要求 backlog、sprint、release 或 CI/CD scaffolding，Codex 会默认使用更轻量的 `lite` profile。

安装后会发生什么?

首次接入时，Codex 会在目标仓库中生成完整的 `SDD/` 工作区、根目录 `AGENTS.md` 和 `README.md` shim，然后阅读它们并帮你创建第一个 parent task，直接进入工作状态。**已有旧版 workflow 时**，Codex 会安全升级现有结构，保留你的任务记录和进度文档，然后继续从上次中断的地方推进。

🤖 给 Codex 的执行指南

如果你是正在读取本 README 的 Codex，请把下面规则当作使用本 skill 的默认执行路径：

1. **先判断场景**
   - 用户要在已有 repo 中初始化、升级或恢复 durable SDD workflow 时，使用 `$codex-sdd-workflow`。
   - 用户没有明确选择 profile 时，先询问 `lite` 还是 `full`；如果没有 backlog / sprint / release / CI-CD 需求，默认推荐 `lite`。
2. **先预览，再写入**
   - 首次接入或不确定覆盖范围时，先执行 dry-run。
   - 不要主动覆盖目标 repo 的根 `README.md` / `AGENTS.md`；只有用户明确要求时才使用强制 root shim 选项。
3. **把脚本当作确定性 API**
   - 创建 task、subtask、ADR、handoff、archive、validate 时，优先调用生成的 `SDD/scripts/` 脚本。
   - 不要手写替代脚本输出，也不要临时改模板格式来绕过 validator。
4. **遵守 SDD 写入边界**
   - 正式任务状态写 `tasks/active/` 或 `tasks/history/`。
   - 共享恢复摘要写 `docs/progress.md`。
   - branch/task 级临时过程写 `state/hot/branches/` 或 `state/hot/tasks/`。
   - 结构化验证记录写 `evidence/records/`。
   - 架构决策写 `adr/records/`。
5. **不要混淆 task card 和 hot-state note**
   - `tasks/active/TASK-*.md` 是正式 task card，必须包含目标、scope、验收标准、验证与 Completion Handoff。
   - `state/hot/tasks/*.md` 只是 task-local scratch note，用来降低共享文件冲突，不能替代 task card。
6. **长会话必须先 checkpoint**
   - 不要把当前目标、已完成内容、未完成内容、下一步、风险、改动文件或验证状态只放在聊天记忆里。
   - 当任务 scope 变大、会话变长、记忆开始模糊、即将关键实现/提交、即将委派 subtasks，或发生过 context 压缩/恢复时，先把 checkpoint 写进 active task、`docs/progress.md` 或 `state/hot/`。
   - checkpoint 至少写清：真实目标、已完成内容、剩余工作、下一步明确动作、风险或 blocker、最近验证状态。
   - 如果 UI 显示 context 使用比例达到或超过 29%，先写 checkpoint，再询问用户：开启压缩，还是切换到新的 session。
7. **完成前必须闭环**

- 任务完成前，必须留下验证 evidence 或说明为什么无法验证。
- 必须留下 next step，或说明项目已经到达终态。
- 必须遵守 `TASK_COMPLETION_GIT_MODE`：默认 `manual` 模式下不自动 commit，而是记录未提交原因和推荐 commit message。

8. **使用 full profile 时保持边界**
   - `full` 是 repo-local agile delivery scaffolding。
   - 若团队已有 Jira / GitHub Issues / Linear，把本地 backlog 当执行镜像和 handoff 载体，不要宣称它是唯一真源。
   - connector hook 当前按 pull-first 使用，不要承诺双向同步。
9. **使用 overlay 时保护 parser contract**
   - 只有用户明确要求组织级模板定制时才使用 `--template-overlay`。
   - overlay 可以增加字段，但不能删除必需标题、checkbox 状态、Acceptance Criteria、Evidence、Completion Handoff 或 Git handoff 字段。

🔀 Profile 选择指南

`lite` —— 精准、轻量、开箱即用 ⭐ 推荐

适用于 **大多数日常开发场景**。覆盖：

- ✅ 持久化开发工作区 & session 恢复
- ✅ 结构化任务卡片 (parent / subtask)
- ✅ 验收标准 & 验证命令
- ✅ 交接文档 & 下一步指引
- ✅ 主 agent + subagent 协作规范

> 🏷️ **公开场景的默认推荐**。绝大多数仓库从 `lite` 开始就够用了。

`full` —— 敏捷交付增强 (Beta)

在 `lite` 的基础上扩展，增加：

- 📋 Backlog 管理
- 🏃 Sprint 记录
- 📦 Release 追踪
- 🔄 CI/CD 导向的目录结构和模板

> ⚠️ **注意**：`full` 提供的是 repo-local 的 scaffolding，不是完整的项目管理平台或 CI/CD 编排系统。真正的 issue tracker、PR 审批、release 环境和流水线，仍需要接入你自己的团队系统。

📂 你的仓库会变成什么样?

以下是 **lite profile** 生成后的完整目录结构（`--no-root-shims` 模式下不会生成根目录的 `AGENTS.md` 和 `README.md`）：

```
your-repo/
├── AGENTS.md                      # 🤖 Codex 的行为契约（启动入口）
├── README.md                      # 📖 项目 shim（指向 SDD workflow）
└── SDD/
    ├── workflow.md                # 📋 Workflow 总指南：启动流程、文件夹角色、脚本用法
    ├── workflow-config.env        # ⚙️ 运行时配置（Git 模式、profile 等）
    ├── docs/                      # 📝 持久化项目记忆
    |   ├── project-brief.md       #    项目简介与背景
    |   ├── process.md             #    流程摘要：当前焦点、活跃任务、归档历史
    |   ├── progress.md            #    进度状态 & 交接面：下一步、风险、建议
    |   ├── architecture.md        #    架构记录
    |   ├── testing.md             #    测试策略、质量门禁、验证命令参考
    |   ├── git-workflow.md        #    分支命名、Conventional Commits、PR 规范
    |   └── workflow-audit.md      #    Workflow 执行审计记录
    ├── tasks/                     # ✅ 任务管理
    |   ├── active/                #    进行中的 task 和 subtask
    |   └── history/               #    已完成并归档的任务
    ├── templates/                 # 📄 模板库（与真实文件物理分离）
    |   ├── tasks/                 #    任务卡片模板
    |   ├── evidence/              #    验证留痕模板（记录测了什么、结果如何）
    |   ├── adr/                   #    架构决策记录模板
    |   └── overlays/              #    受控模板 overlay 记录
    ├── adr/records/               # 🏛️ 架构决策记录（ADR）
    ├── evidence/records/          # 🔍 实际的验证记录存档（由模板生成）
    ├── state/                     # 🔥 branch/task 级热状态便签
    |   ├── hot/                   #    当前高频更新的局部状态，不是 task card
    |   |   ├── branches/          #    branch-local 便签
    |   |   └── tasks/             #    task-local 便签
    |   └── history/               #    归档后仍值得保留的热状态记录
    └── scripts/                   # 🛠️ 日常工作脚本
        ├── session-brief.sh / .ps1  #    Session 恢复摘要
        ├── validate-sdd.sh / .ps1   #    Workflow 合规检查
        ├── new-task.sh / .ps1       #    创建新任务
        ├── new-subtask.sh / .ps1    #    创建子任务
        ├── archive-task.sh / .ps1   #    归档已完成任务
        ├── new-adr.sh / .ps1        #    创建架构决策记录
        └── handoff-template.sh/.ps1 #    生成交接文档
```

> 📎 **关键文件联动关系**
>
> **任务生命周期链**：`tasks/active/` 中创建任务 → 分支或 task 级细节写入 `state/hot/` → 开发完成 → 归档到 `tasks/history/` → 同步更新 `docs/process.md`（长期流程摘要）和 `docs/progress.md`（共享恢复摘要）
>
> **验证留痕链**：参考 `docs/testing.md` 的命令和门禁 → 执行验证 → 基于 `templates/evidence/` 模板填写记录 → 保存到 `evidence/records/` → `validate-sdd --strict` 自动检查是否有留痕
>
> 简单来说：`process.md` 是项目的"长期记忆"，`progress.md` 是共享恢复摘要，`tasks/` 是正式 task card，`state/hot/` 是 branch/task 级临时便签——四者共同构成 Codex 每次恢复上下文的信息来源。
>
> **注意**：`tasks/active/` 里的 `TASK-*.md` / `SUBTASK-*.md` 是正式任务卡，包含目标、scope、验收标准和 Completion Handoff；`state/hot/tasks/` 只是跟 task 同名的短期实现便签，用来记录高频细节，不能替代 task card。

使用 **full profile** 时，还会额外生成：

```
SDD/
├── backlog/                          # 📋 需求池 / Backlog 管理
├── sprints/                          # 🏃 Sprint 记录
├── releases/                         # 📦 Release 追踪
├── automation/                       # 🔄 CI/CD 自动化配置
├── scripts/
|   └── sync-external-items.sh/.ps1   #     外部 issue source 的 pull-only connector hook
└── docs/
    ├── agile-delivery.md             #     敏捷交付流程说明
    └── ci-cd.md                      #     CI/CD 集成指南
```

`full` 还会在 `workflow-config.env` 里预留：

- `EXTERNAL_ISSUE_SOURCE=none`
- `CONNECTOR_MODE=pull-only`

它们用来表达一个很重要的边界：`full` 是 repo-local 的执行与交付 scaffolding，不是完整的外部系统同步平台。

🧭 日常使用完全指南（AI 原生交互）

> ✨ **核心心智：动嘴不动手**
> 你会在文档和目录中看到大量的环境脚本（如 `new-task.sh`、`archive-task.sh` 等）。
> **请注意：设计这些并不是要求你手动去敲的！** 它们是暴露给 Codex 调用的**底层确定性 API**。
> 本工作流提倡 **自然语言驱动**，你全程无需去记命令行，像跟同事说话一样驱使 Codex，它会在后台自动为你挂载、执行脚本并反馈结果。对于极客用户，你当然也可以随时在终端手动调用这些同名脚本。

1️⃣ 每次开始工作——恢复上下文

当你新开一个 Session 切入项目时：
🗣️ **你只需要说**：*"给我一份当前的项目状态与进度简报"*

> 🤖 **底层原理**：Codex 会自动为你阅读 `AGENTS.md` → `SDD/docs/process.md`，并在后台执行 `./SDD/scripts/session-brief.sh` 读取最新进度快照，瞬间从你上一次离开的地方接头。

2️⃣ 创建任务——动手之前先建卡

不要让 Codex 漫无目的地直接改代码，先用一句话让它建立护栏：
🗣️ **你只需要说**：*"我想开始做用户登录模块，先帮我建个主任务然后简要拆分一下"*

> 🤖 **底层原理**：Codex 将调用内部工具链 `./SDD/scripts/new-task.sh` 安全生成一张带有验收标准和 Scope 界定的卡片。建卡是为了提供明确的终点和验证条件，遏制 AI 的代码漂移。

3️⃣ 开发中——按 scope 推进

一句话让它干活即可：
🗣️ **你只需要说**：*"根据刚才的登录卡开始执行，需要建子任务就建"*

- Codex 会在开发过程中根据它的判断调用 `new-subtask.sh` 并行推进。
- 如果一个任务自然拆成多个独立可验证的工作单元，Codex 会自主完成架构拆解与追踪。

- 需要记录大量 branch/task 级细节时，Codex 会优先写入 `SDD/state/hot/`，而不是把 `progress.md` 写成共享冲突热点。

4️⃣ 完成任务——检查、验收与归档

功能写完后，无需手动清场：
🗣️ **你只需要说**：*"登录逻辑开发完了，请进行合规检查，帮我做验证留痕并把它归档"*

> 🤖 **底层原理**：Codex 将为你流水线执行以下三步闭环：
>
> 1. 参考 `docs/testing.md` 帮你跑测。
> 2. 执行工具 `./SDD/scripts/validate-sdd.sh --strict` 并向 `evidence/records/` 中写入实测留痕日志。
> 3. 最终通过 `./SDD/scripts/archive-task.sh` 安全清理工作区状态。

5️⃣ 交接 / 换人 / 下次继续

下班前或者需要把工作在明日留给同事：
🗣️ **你只需要说**：*"我要下线了，帮我生成交接文档"*

> 🤖 **底层原理**：Codex 获取交接 API `handoff-template.sh` 权限，生成交接文档并沉淀当前状态、未完成项、风险点以及下一步建议，并在内部按 `TASK_COMPLETION_GIT_MODE` 模式妥善处理你的 commit。没有任何历史包袱，明天谁接手都不抓瞎。

📌 关键文件速查

想做什么	看哪个文件
了解项目背景和目标	`SDD/docs/project-brief.md`
查看当前焦点和进度	`SDD/docs/process.md` + `progress.md`
查看 branch/task 级临时细节	`SDD/state/hot/branches/` + `SDD/state/hot/tasks/`
查阅架构决策	`SDD/adr/records/`
查找历史任务和经验	`SDD/tasks/history/`
查看验证证据	`SDD/evidence/records/`
理解 Git 协作规范	`SDD/docs/git-workflow.md`
审计 workflow 执行质量	`SDD/docs/workflow-audit.md`

⚠️ 使用注意事项

Codex首次接入

- 🔍 **先预览再执行**：首次推荐使用 `--dry-run` 预览生成内容，确认无误后再正式写入
- 📝 **第一时间填写 `project-brief.md`**：这是 Codex 理解你项目的起点，越早填写越好
- 🔀 **已有 README / AGENTS.md 不会被覆盖**：除非你明确使用 `--force-root-shims`

Codex日常使用

- ✅ 用 `[ ]` 标记未完成项，`[x]` 标记已完成项——Codex 和脚本都依赖这个约定
- 📁 活跃任务放 `tasks/active/`，完成后及时归档到 `tasks/history/`，保持工作区整洁
- 🔥 高频实现便签放 `state/hot/`，共享恢复摘要才写入 `docs/progress.md`
- 🚫 **不要手动修改 `templates/` 目录中的模板**——它们是生成新卡片的源，修改会影响后续创建
- 🔄 `process.md` 是长期维护的流程摘要，`progress.md` 是共享恢复摘要，`state/hot/` 是局部便签——不要混淆

Git 相关

- 默认 `TASK_COMPLETION_GIT_MODE=manual`：Codex **不会自动帮你 commit**
- 每个任务闭环时会记录：commit 状态、未提交原因、推荐 commit message
- Subagent 永远不会拥有 commit 权限——集成和 Git 闭环由主 agent 负责
- 如需自动提交，可在 `workflow-config.env` 中将模式改为 `auto`（请谨慎使用）

多人协作

- 每个 subtask 有明确的 scope 边界，文件所有权保持不相交
- 主 agent 负责集成和最终验证，subtask 只报告变更和风险
- 任务卡片和交接文档是协作的通信契约——**请认真填写**
- 多分支或多人并行时，优先把细节写到 branch/task 级 hot state，降低 `progress.md` 的 merge 冲突概率

模板 Overlay

- 默认不需要 overlay，内置模板已经覆盖大多数场景
- 如需组织级定制，可使用 `--template-overlay /path/to/overlay`
- overlay 只能覆盖受支持模板，并且必须保留 parser contract 所需标题、checkbox 字段和 handoff 字段
- 合法 overlay 会在 `SDD/templates/overlays/active/manifest.json` 中留下记录

🛡️ 安全设计哲学

我们在每一个可能造成破坏的环节都设置了 **安全默认值**：

行为	默认状态	说明
覆盖已有 workflow 目录	❌ 不覆盖	使用 `--force` 明确选择才会覆盖
替换根目录 `README.md` / `AGENTS.md`	❌ 不替换	需要 `--force-root-shims` 才会替换
Git 自动提交	❌ 不自动	默认 `TASK_COMPLETION_GIT_MODE=manual`
Subagent 提交权限	❌ 仅记录	最终 commit 和集成始终由主 agent 负责

> 🔒 **承诺**：不会在 issue、PR、测试 fixture、验证报告或示例中提交 API key、credential、私有路径或客户数据。

✅ 验证 & 测试

运行完整测试套件

```
python -m pip install PyYAML
python -m unittest discover -s tests -p "test_*.py" -v
python scripts/run_skill_validation.py --skip-tests
```

生成本地验证报告

```
python scripts/run_skill_validation.py --print-forward-prompts --report reports/skill-validation.md
```

> 验证脚本会优先使用已安装的 `skill-creator` 的 `quick_validate.py`；在干净的 CI 环境中，如果没有 system skill，会自动回退到本地 fallback validator——**开箱即用，无需额外依赖**。

🤝 参与贡献

欢迎提交 Issue 和 PR！贡献前请阅读 `CONTRIBUTING.md`。

最低验证要求（PR 前必须通过）：

```
python -m pip install PyYAML
python -m unittest discover -s tests -p "test_*.py" -v
python scripts/run_skill_validation.py --skip-tests
```

涉及 skill 触发语义、profile 行为、bootstrap 输出或验证报告的变更，请同时更新 `reports/skill-validation.md`。

🔐 安全问题

安全漏洞请 **私下联系维护者** 报告，不要公开可利用的细节。详见 `SECURITY.md`。

License

MIT License — 自由使用、修改和分发。

让每个 session 都从上一次结束的地方开始，而不是从零开始。

Codex SDD Workflow · Built for durable AI-assisted engineering.